LINKED LIST

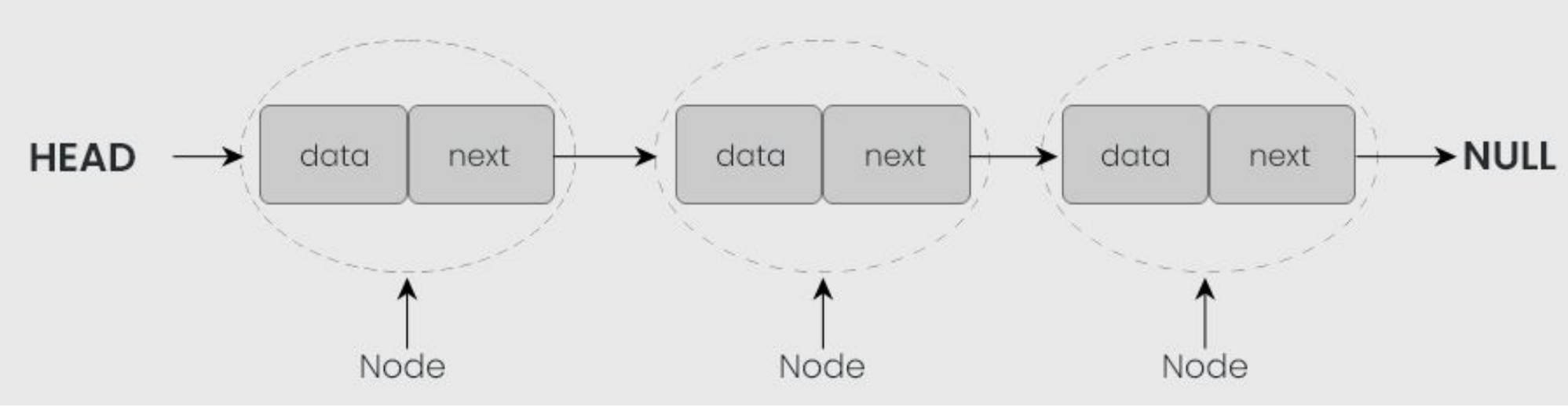
Nazirul Hasan
Lecturer, CSE, KUET

TYPES OF LINKED LIST

- Singly linked list
- Doubly linked list
- Circular linked list
- Circular doubly linked list

WHAT IS LINKED LIST?

- A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers as shown in the below image:

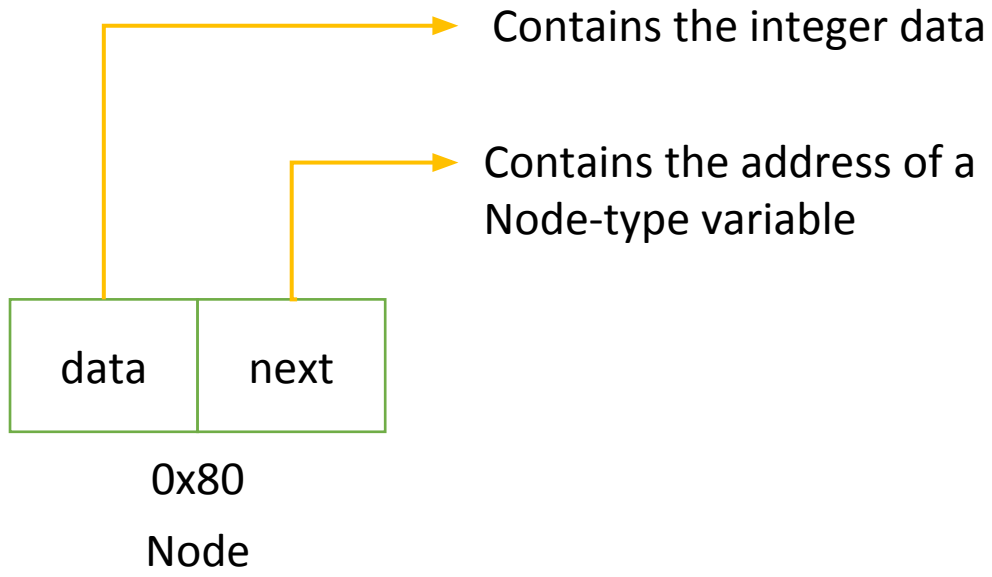


WHAT IS LINKED LIST?

- In simple words, Linked List forms a series of connected nodes, where each node stores the data and the address of the next node.

A NODE CREATION

```
// A Single linked list node

class Node {

public:

    int data;

    Node* next;

};
```



You can create a linked list by creating several of these nodes and by connecting them in a way where each node stores the data and the address of the next node.

IMPLEMENTING LINKED LIST USING CLASS

```
class Node {
public:
    int data;
    Node* next;

    // Default constructor
    Node()
    {
        data = 0;
        next = NULL;
    }

    // Parameterised Constructor
    Node(int data)
    {
        this->data = data;
        this->next = NULL;
    }
};
```

```
// Linked list class to
// implement a linked list.
class Linkedlist {
    Node* head;

public:
    // Default constructor
    Linkedlist() { head = NULL; }

    // Function to insert a
    // node at the end of the
    // linked list.
    void insertNode(int);

    // Function to print the
    // linked list.
    void printList();

    // Function to delete the
    // node at given position
    void deleteNode(int);
};
```

IMPLEMENTING LINKED LIST USING CLASS

- You can create a linked list by writing -
  - LinkedList list; or, LinkedList list = LinkedList();
  - LinkedList *list = new LinkedList();

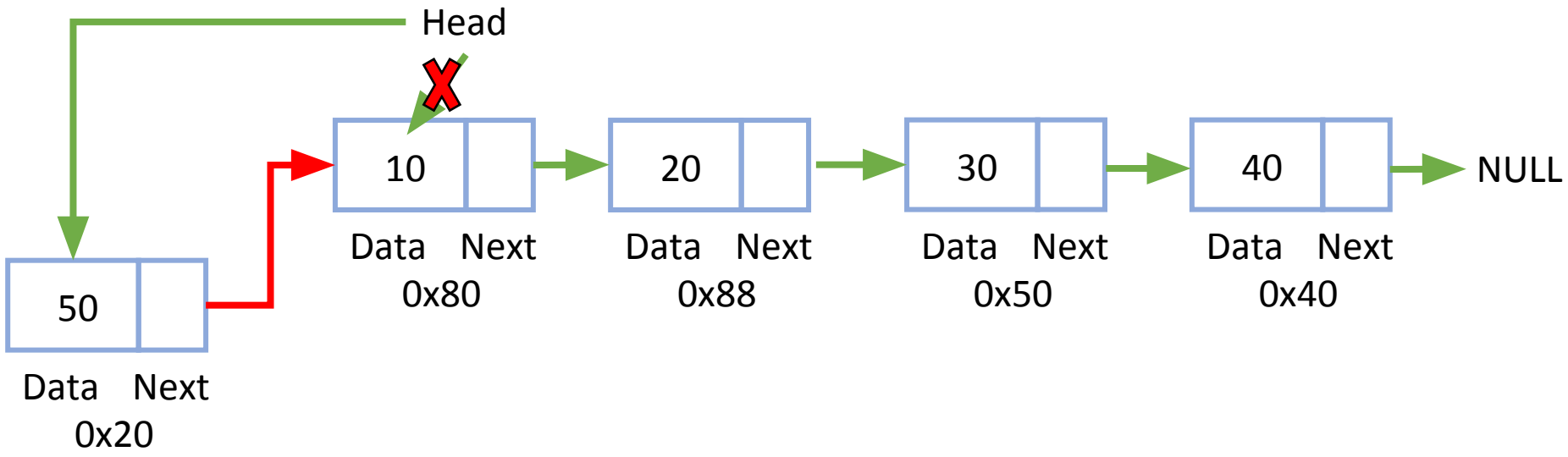
- You can create a node by writing –
  - Node newnode;
  - Node newnode = Node(); // default constructor
  - Node newnode = Node(10); // parameterized constructor
  - Node *newnode = new Node(); // default constructor
  - Node *newnode = new Node(10); // parameterized constructor

INSERTION IN LINKED LIST

Three possible scenarios

1. At the front of the linked list
2. After a given node.
3. At the end of the linked list.

HOW TO INSERT A NODE AT THE FRONT OF A LINKED LIST



- Make the first node of the Linked List linked to the new node.
- Remove the head from the original first node of Linked List.
- Make the new node as the Head of the Linked List.

HOW TO INSERT A NODE AT THE FRONT OF A LINKED LIST

- Now, make a public method name “insertAtFront()” to insert a node at the beginning of a linked list.

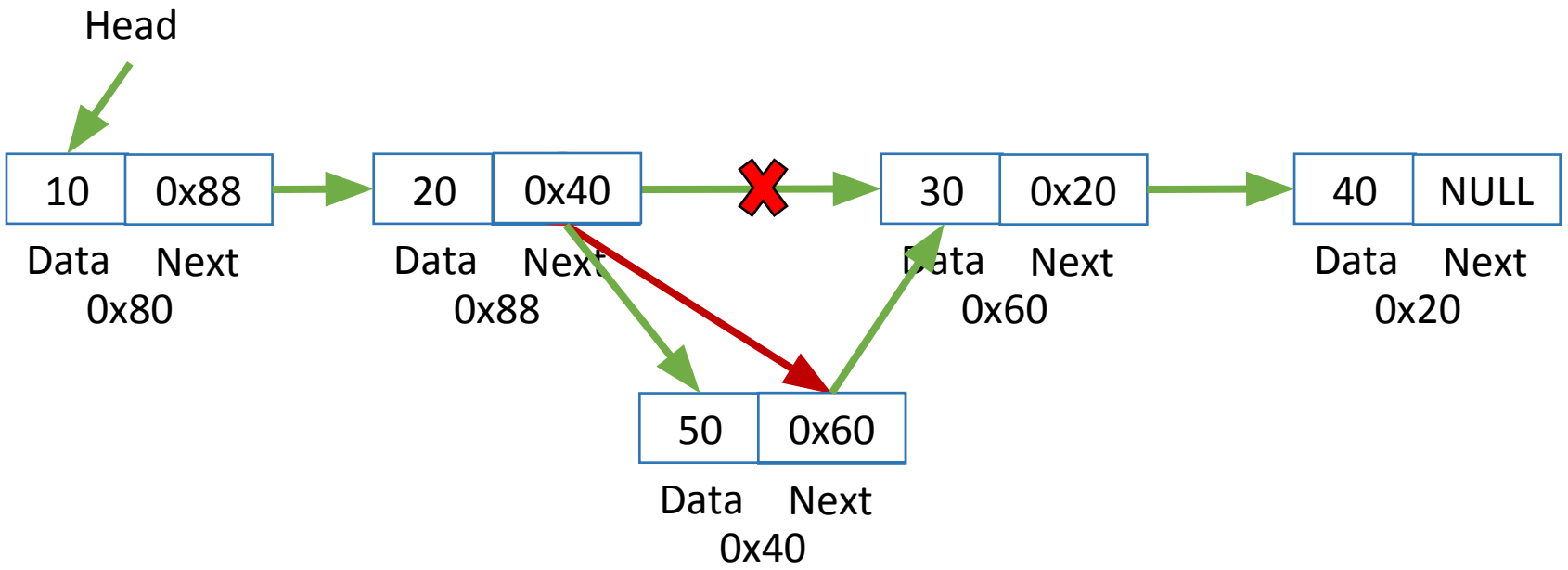
```
// 2. Insert at first
void insertFirst(int value) {
    Node* newNode = new Node(value);
    newNode->next = head;
    head = newNode;
}
```

COMPLEXITY ANALYSIS

- Time Complexity: O(1), We have a pointer to the head and we can directly attach a node and change the pointer. So the Time complexity of inserting a node at the head position is O(1) as it does a constant amount of work.

- Auxiliary Space: O(1)

HOW TO INSERT A NODE AFTER A GIVEN NODE IN THE LINKED LIST



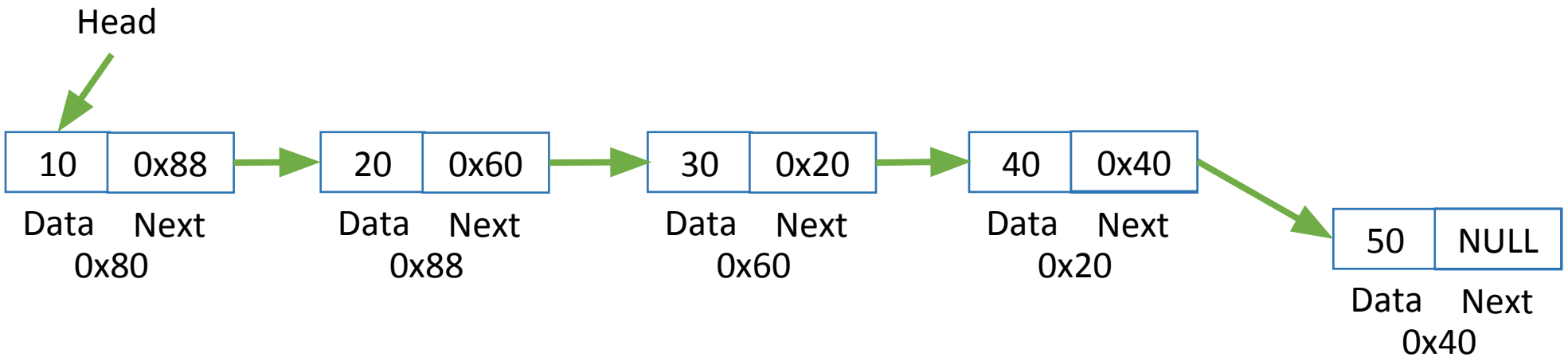
HOW TO INSERT A NODE AFTER A GIVEN NODE IN THE LINKED LIST

- Now, make a public method name “insertAfterGivenNode()” to insert a node after a given node in the linked list.

```
void insertAfter(int key, int value) {
    Node* temp = head;

    while(temp != NULL && temp->data != key)
        temp = temp->next;

    if(temp == NULL) {
        cout << "Value not found\n";
        return;
    }

    Node* newNode = new Node(value);
    newNode->next = temp->next;
    temp->next = newNode;
}
```

Time Complexity: O(n)
Auxiliary Space: O(1)

HOW TO INSERT A NODE AT THE END OF LINKED LIST



- Go to the last node of the Linked List
- Change the next pointer of last node from NULL to the new node
- Make the next pointer of new node as NULL to show the end of Linked List

COMPLEXITY ANALYSIS

- Time complexity: O(N), where N is the number of nodes in the linked list. Since there is a loop from head to end, the function does O(n) work.
  - This method can also be optimized to work in O(1) by keeping an extra pointer to the tail of the linked list/
- Auxiliary Space: O(1)

HOW TO INSERT A NODE AT THE END OF LINKED LIST

```cpp
// Function to insert a new node.
void Linkedlist::insertNode(int data)
{
    // Create the new Node.
    Node* newNode = new Node(data);

    // Assign to head
    if (head == NULL) {
        head = newNode;
        return;
    }

    // Traverse till end of list
    Node* temp = head;
    while (temp->next != NULL) {

        // Update temp
        temp = temp->next;
    }

    // Insert at the last.
    temp->next = newNode;
}
```

FUNCTION TO PRINT THE NODES OF THE LINKED LIST

```
// Function to print the
// nodes of the linked list.
void Linkedlist::printList()
{
    Node* temp = head;

    // Check for empty list.
    if (head == NULL) {
        cout << "List empty" << endl;
        return;
    }

    // Traverse the list.
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
}
```

SEARCH AN ELEMENT IN A LINKED LIST (ITERATIVE APPROACH)

- Follow the below steps to solve the problem:

- Initialize a node pointer i.e temp= head or current = head.
- Do following while current is not NULL
- If the current value (i.e., current->key) is equal to the key being searched return true.
- Otherwise, move to the next node (current = current->next).
- If the key is not found, return false

Time Complexity: O(N), Where N is the number of nodes in the LinkedList
Auxiliary Space: O(1)

SEARCH AN ELEMENT IN A LINKED LIST (ITERATIVE APPROACH)

Time Complexity: O(N), Where N is the number of nodes in the LinkedList

Auxiliary Space: O(1)

```
void searchValue(int value) {
    int position = 1;
    Node* temp = head;

    while(temp != NULL) {
        if(temp->data == value) {
            cout << "Value found at position: " << position << endl;
            return;
        }
        temp = temp->next;
        position++;
    }

    cout << "Value not found\n";
}
```

AN ITERATIVE APPROACH FOR FINDING THE LENGTH OF THE LINKED LIST:

- Follow the given steps to solve the problem:

- Initialize count as 0
- Initialize a node pointer, current = head.
- Do following while current is not NULL
- current = current -> next
- Increment count by 1.
- Return count

```
void printSize() {
    int count = 0;
    Node* temp = head;

    while(temp != NULL) {
        count++;
        temp = temp->next;
    }

    cout << "Size of list: " << count << endl;
}
```

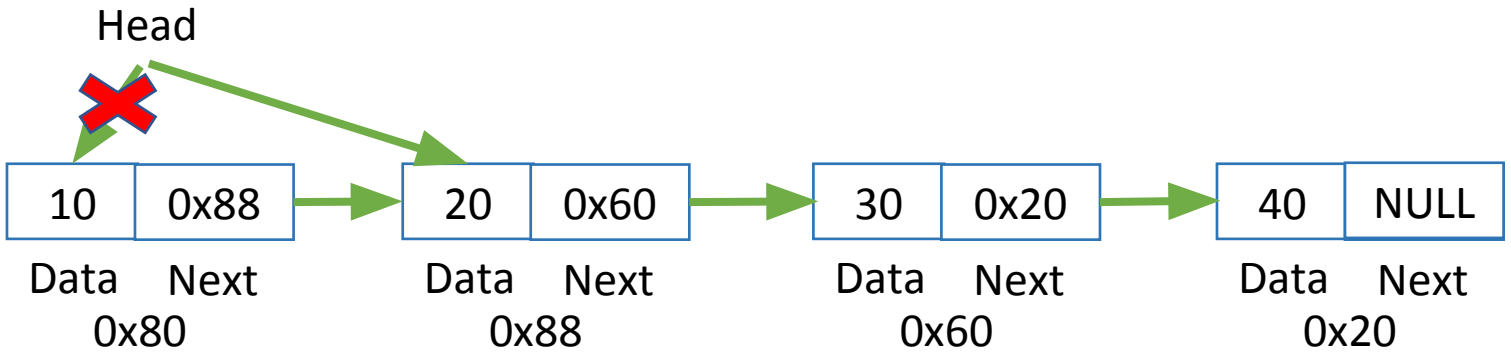
Time complexity: O(N), Where N is the size of the linked list
Auxiliary Space: O(1), As constant extra space is used.

DELETION IN LINKED LIST

- You can delete an element in a list from:

- Beginning
- End
- Middle

DELETE FROM BEGINNING



Point head to the next node i.e. second node

temp = head

head = head->next

Make sure to free unused memory

free(temp); or delete temp;

Time Complexity: O(1)

Auxiliary Space: O(1)

DELETE FROM BEGINNING

- Now, make a public method name “deleteFromBeginning()” to delete the first node in the linked list.

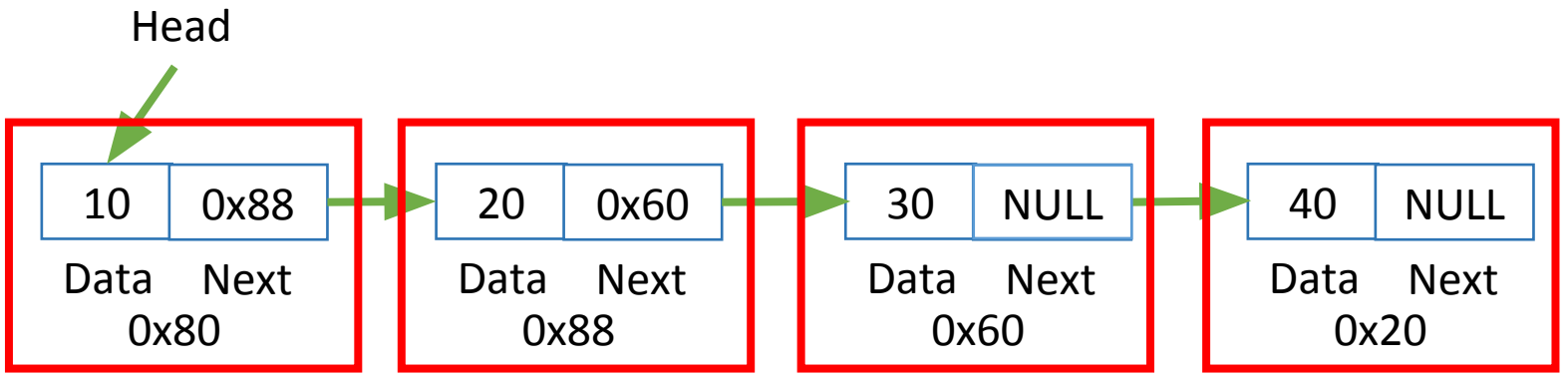
```
void deleteFirst() {
    if(head == NULL) return;

    Node* temp = head;
    head = head->next;
    delete temp;
}
```

DELETE FROM END

Point head to the previous element i.e. last second element

Change next pointer to null

```
Node *end = head;
Node *prev = NULL;
while(end->next != NULL)
{
    prev = end;
    end = end->next;
}
prev->next = NULL;
```



Time Complexity: O(n)

Auxiliary Space: O(1)

Make sure to free unused memory

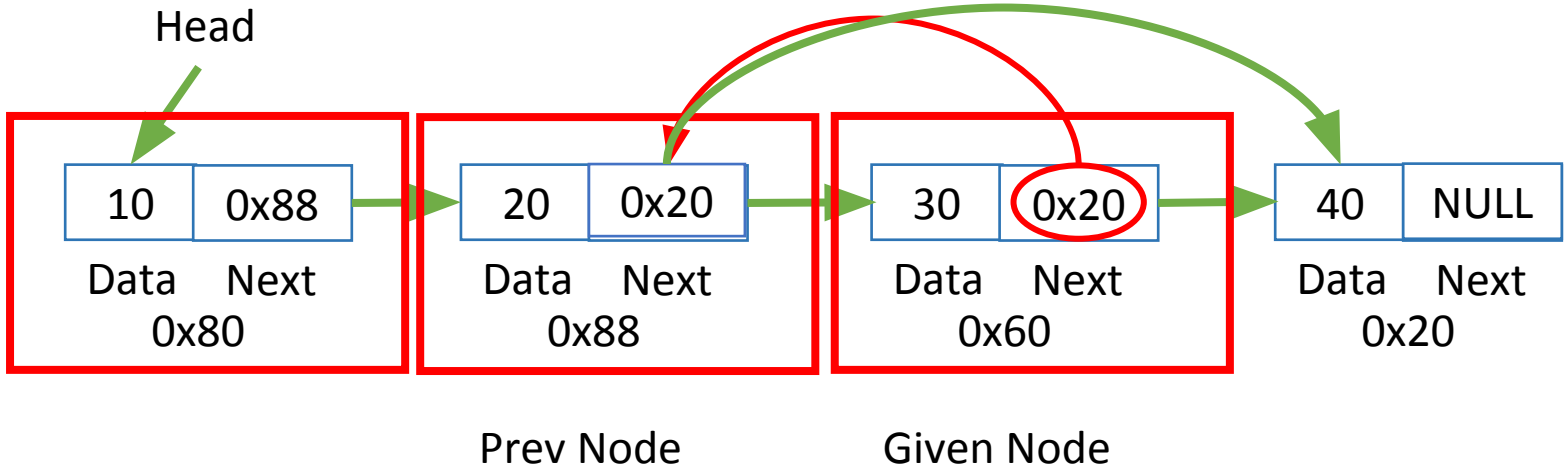
free(end); or delete end;

DELETE FROM END

- Now, make a public method name “deleteFromEnd()” to delete the last node in the linked list.

```
void deleteLast() {
    if(head == NULL) return;

    if(head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* temp = head;
    while(temp->next->next != NULL)
        temp = temp->next;

    delete temp->next;
    temp->next = NULL;
}
```

DELETE FROM MIDDLE



1. Find Given node, temp := Given node
2. Set link of the previous node to link of given nodes link

   e.g. prev.next=Given_node.next
3. free(temp), or delete temp

Time Complexity: O(n)
Auxiliary Space: O(1)

DELETE A LINKED LIST NODE AT A GIVEN POSITION

```
// Function to delete the
// node at given position
void Linkedlist::deleteNode(int nodeOffset)
{
    Node *temp1 = head, *temp2 = NULL;
    int ListLen = 0;

    if (head == NULL) {
        cout << "List empty." << endl;
        return;
    }

    // Find length of the linked-list.
    while (temp1 != NULL) {
        temp1 = temp1->next;
        ListLen++;
    }
```

```
// Check if the position to be
// deleted is greater than the length
// of the linked list.
if (ListLen < nodeOffset) {
    cout << "Index out of range"
        << endl;
    return;
}
// Declare temp1
temp1 = head;

// Deleting the head.
if (nodeOffset == 1) {

    // Update head
    head = head->next;
    delete temp1;
    return;
}
```

DELETE A LINKED LIST NODE AT A GIVEN POSITION

```
    // Traverse the list to
    // find the node to be deleted.
    while (nodeOffset-- > 1) {

        // Update temp2
        temp2 = temp1;

        // Update temp1
        temp1 = temp1->next;
    }

    // Change the next pointer
    // of the previous node.
    temp2->next = temp1->next;

    // Delete the node
    delete temp1;
}
```

DELETE A LINKED LIST NODE A GIVEN VALUE

```
// 8. Delete given value
void deleteValue(int value) {
    if(head == NULL) return;

    if(head->data == value) {
        deleteFirst();
        return;
    }

    Node* temp = head;
    while(temp->next != NULL && temp->next->data != value)
        temp = temp->next;

    if(temp->next == NULL) {
        cout << "Value not found\n";
        return;
    }

    Node* del = temp->next;
    temp->next = del->next;
    delete del;
}
```

REFERENCES

- Schaum's Outlines Data Structures with C by Seymour Lipscutz.
- https://www.geeksforgeeks.org/data-structures/linked-list/
- https://www.tutorialspoint.com/data_structures_algorithms/linked_list_algorithms.htm
- https://www.programiz.com/dsa/linked-list